

核心问题：由于答辩会分阶段测试，我们需要保证三部分的输出都是独立的

1. 隔离的输入和输出：

我们的目录组织一般是这样：

```
Finallab/  
├─ lex/  
│   ├── 词法解析器  
│   ├── 测试用例  
│   └─ 词法输出  
├─ Syntax/  
│   ├── 语法解析器  
│   └─ 语法输出  
├─ compile_ir/  
└─ 自动化脚本和文档
```

可以看到词法分析器的**工作目录**为lex，语法分析器的**工作目录**为 Syntax，输出和运行都在各自的工作目录下，这是因为在设计的时候考虑到二者需要能独立工作。

Syntax将lex的输出作为输入。

- **这代表什么：**也就是说Syntax不会更改lex下的输出！需要保证阶段性的成果是可展示的！

2.那么对于中间代码，我们如何隔离呢

- 首先明确中间代码部分的输入：

到底是语法生成树tree，还是syntax序列，还是直接跳过了这些，使用了词法的lex序列。

基本可以确定的是：**一定是语法树！**

但是目前的目录组织：**很混乱！**对于中间代码，我们不知道它**使用了哪个文件输入**

想法：**把工作架构改为这样的目录组织**

```
Finallab/  
├─ lex/  
│   ├── 词法解析器  
│   ├── 测试用例  
│   └─ 词法输出  
├─ Syntax/  
│   ├── 语法解析器  
│   └─ 语法输出  
├─ compile_ir/      !!!  
│   ├── 助教提供的原始部件等等  
│   ├── 中间代码生成器  ##我们写的IRGenerator.cpp等文件  !  
│   └─ IRmain.cpp
```

```

|   └─ IRGenerator.cpp
|   └─ 中间代码输出output    ##生成的.ll文件    !
└─ 自动化脚本和文档

```

也就是说：中间代码在工作目录compile_ir下运行输出，不会对前面的**词法、语法输出文件**进行更改。

是哪些文件呢：

- 首先是**out类**文件：这类文件其实是log文件，日志记录性质的，是与中间代码完全无关的！**既不应该被作为输入，也不应该被modify。**
- 接下来是**syntax类**文件：这类文件是**作业中有格式要求的输出**，可以被作为输入，但不能被**modify**
- 最后是**tree类**文件：这类文件**可以被作为输入，也可以被modify**

3.所以，需要做什么呢

- 更改运行目录：使得中间代码生成器通过读入**Syntaxer**的输出，在compile_ir目录下独立工作，像下面这样组织。

```

└─ compile_ir/    ! ! !
|   └─ 助教提供的原始部件等等
|   └─ 中间代码生成器    ##我们写的IRGenerator.cpp等文件    !
|   └─ 中间代码输出output    ##生成的.ll文件    !

```

- 保留语法分析器**Syntaxer**的输出：尤其是**out类**和**syntax类**文件，不可以被更改。这部分代码位于 `ReductionSequenceLogger.cpp` 和 `Parser.cpp` 文件中。
- 为中间代码生成器提供**单独的main入口**：目前中间代码生成器集成在语法器的main入口中，我们需要为它提供一个单独的运行窗口，也就是说，它可以等语法输出结束后在被单独调用，而不是和语法的输出一起产生。大概这样组织：

```

└─ compile_ir/    ! ! !
|   └─ 助教提供的原始部件等等
|   └─ 中间代码生成器    ##我们写的IRGenerator.cpp等文件    !
|       └─ IRmain.cpp    ##ir的main入口
|       └─ IRGenerator.cpp
|   └─ 中间代码输出output    ##生成的.ll文件    !

```